HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Design and Development of a Full-Stack E-Commerce System with Customer Portal and Management Dashboard

Phạm Trường Sang – 20225996

Supervisor: *Dr. Trần Hải Anh*

ONE LOVE. ONE FUTURE.

Outline

1. Motivation & Objectives
2. System Analysis & Design
3. Solutions & Implementation Results
4. Conclusion & Future Work

1. Motivation & Objectives

1. Motivation & Objectives

- **Background**: E-commerce is reshaping modern retail, setting new standards for shopping experiences.

- **Domain Challenges:**
  - **Complex Data:** Tech products have diverse, category-specific specifications.
  - **Management**: High risks of inconsistent data across products, inventory, orders, and payments if handled manually.

- **Objectives:**
  - **System Development**: Develop a full-stack e-commerce system featuring both a Customer Portal and a Management Dashboard
  - **Integrated Workflow**: Connect customer purchasing activities seamlessly with internal store operations to ensure data consistency.

HUST

hust.edu.vn fb.com/dhbkhn

2. System Analysis & Design

2.1. Functional Overview

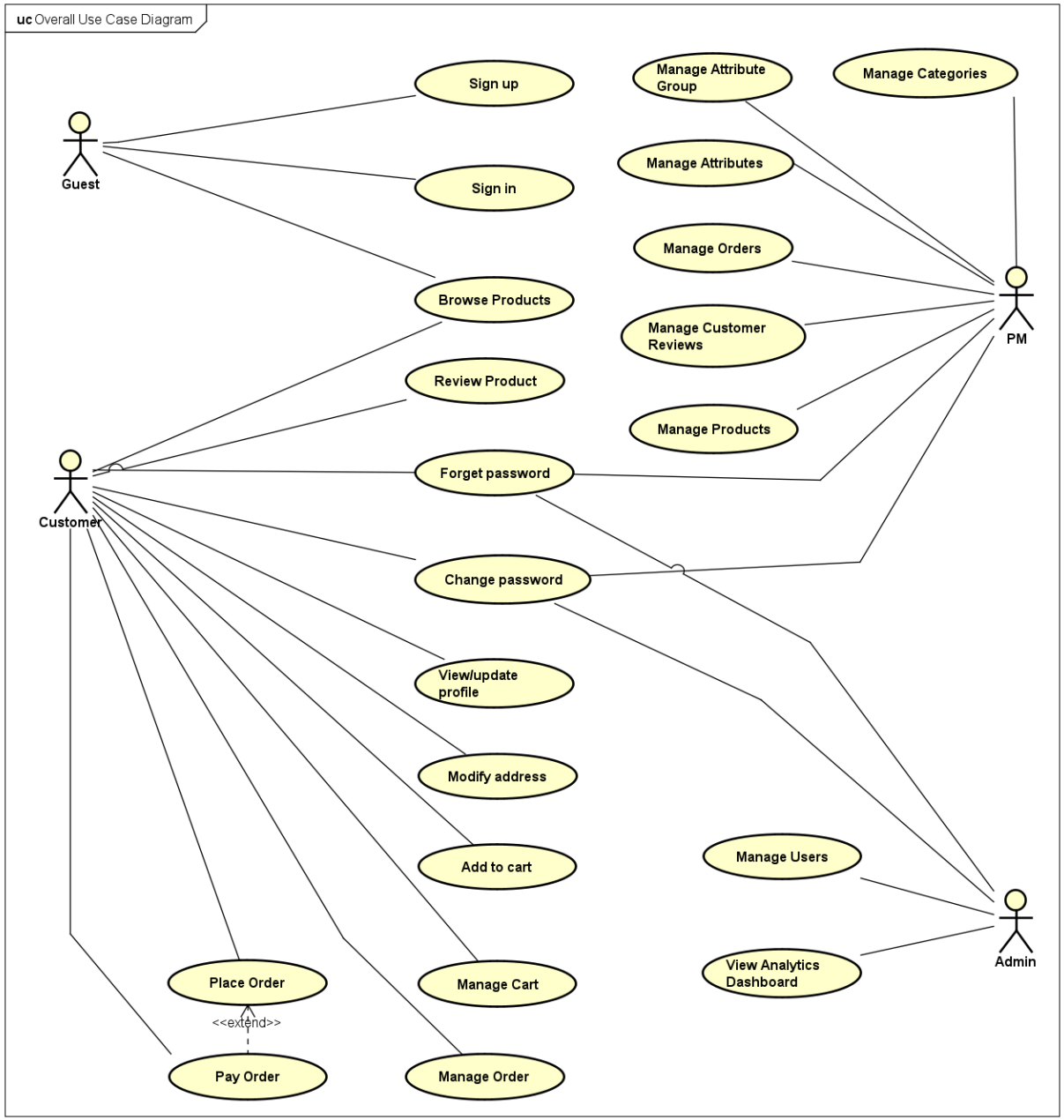
2.2. Core Workflows

2.3. Architectural Design

2.4. Database Design

2.1. Functional Overview

- **4 actors**
- **21 main use cases**
- **2 functional sides:**
  - Customer Portal
  - Management Dashboard

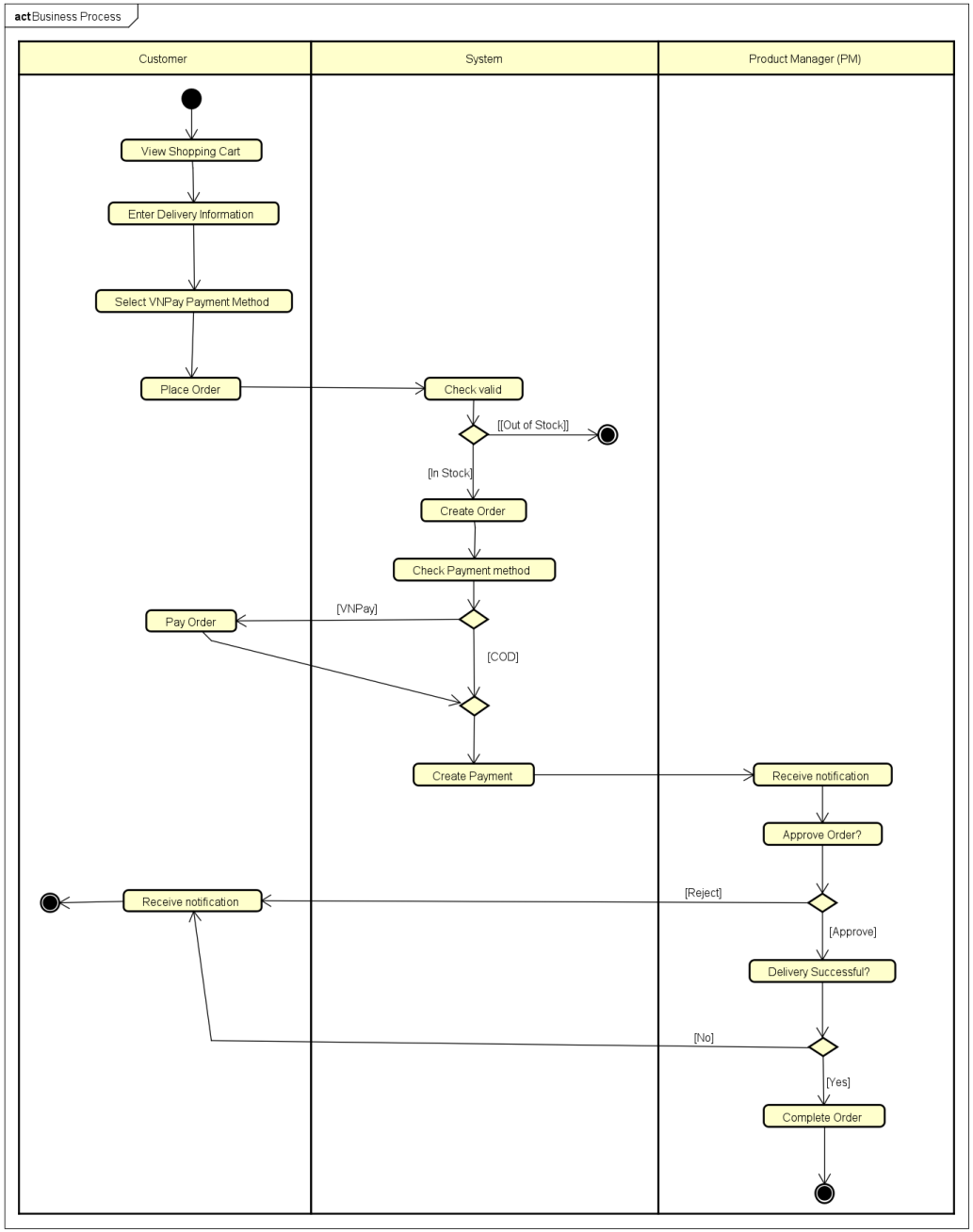


2.2. Core Workflows

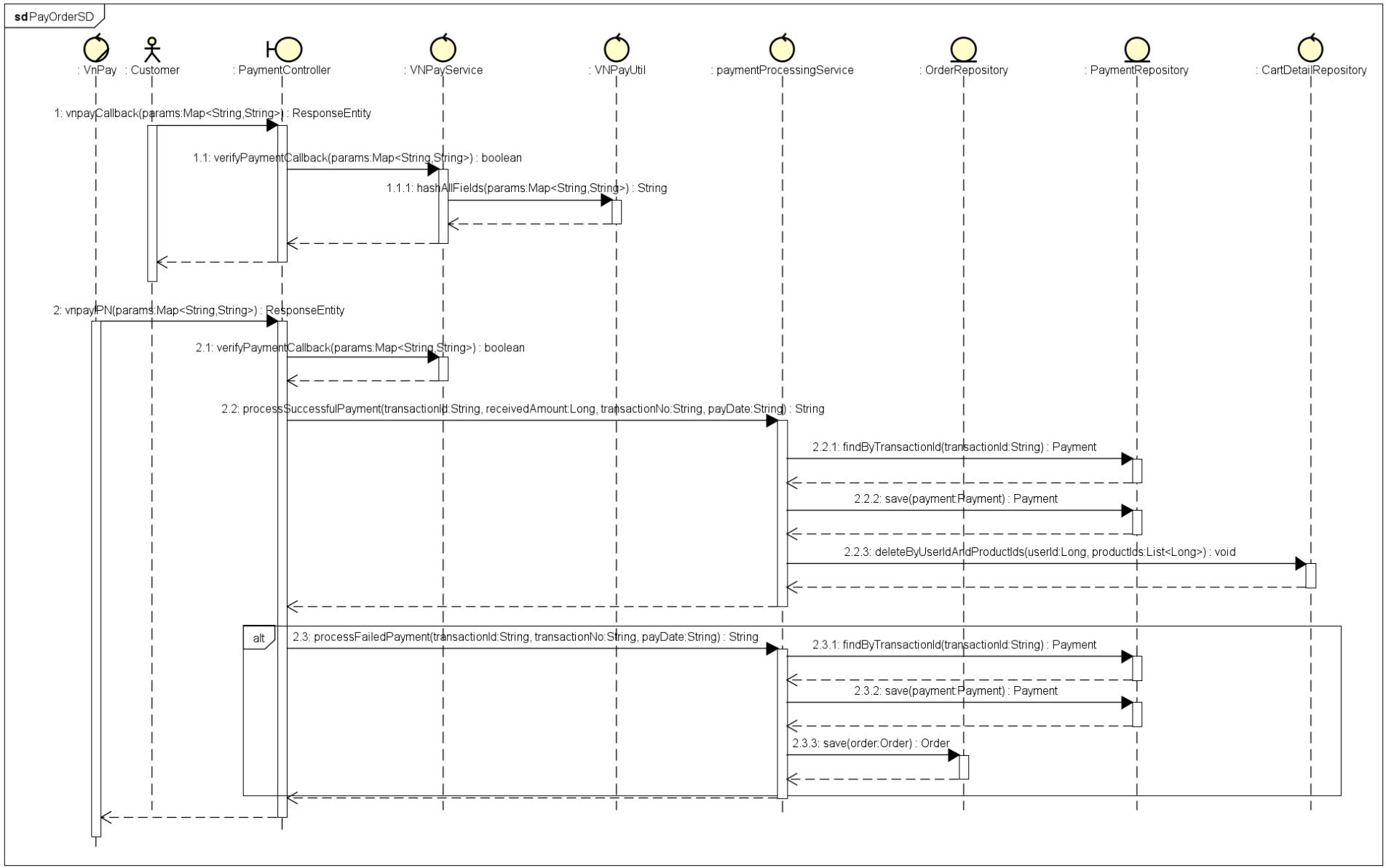
Business Workflow Overview :

- **Checkout:** Customer reviews the cart and places the order.
- **Place Order:** Validate cart and create the order.
- **Payment:** The system handles either COD or VNPay payment.
- **Fulfillment:** The Product Manager processes the confirmed order until completion.

→ The workflow maintains payment correctness and inventory consistency throughout the order lifecycle.

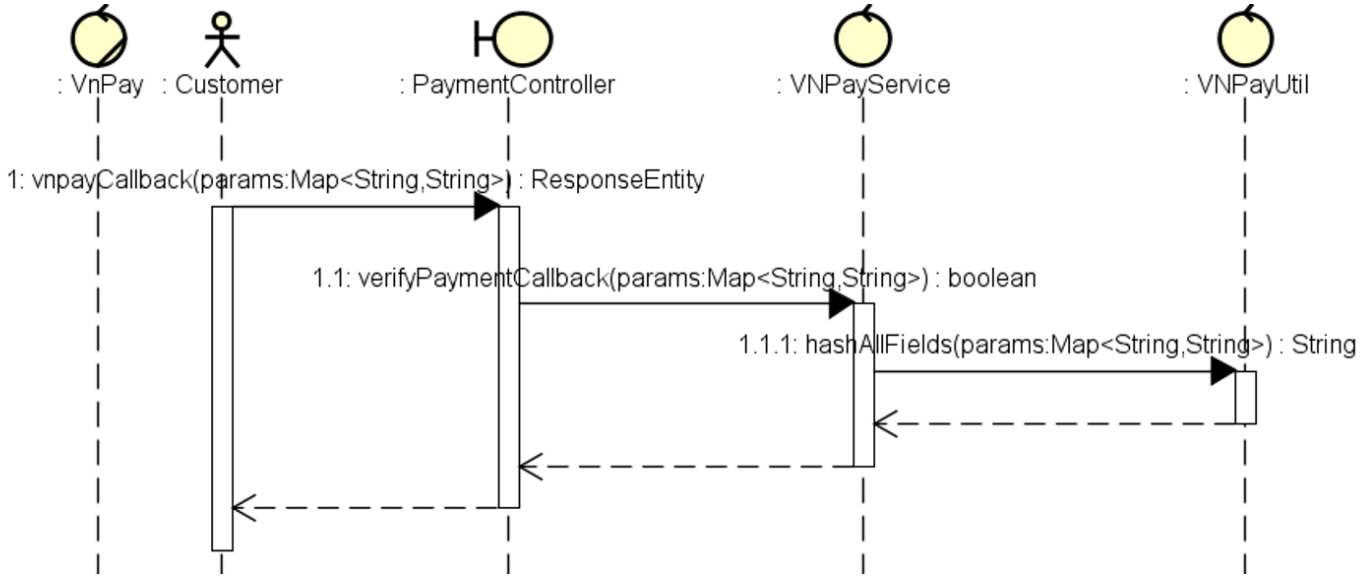


2.2. Core Workflows



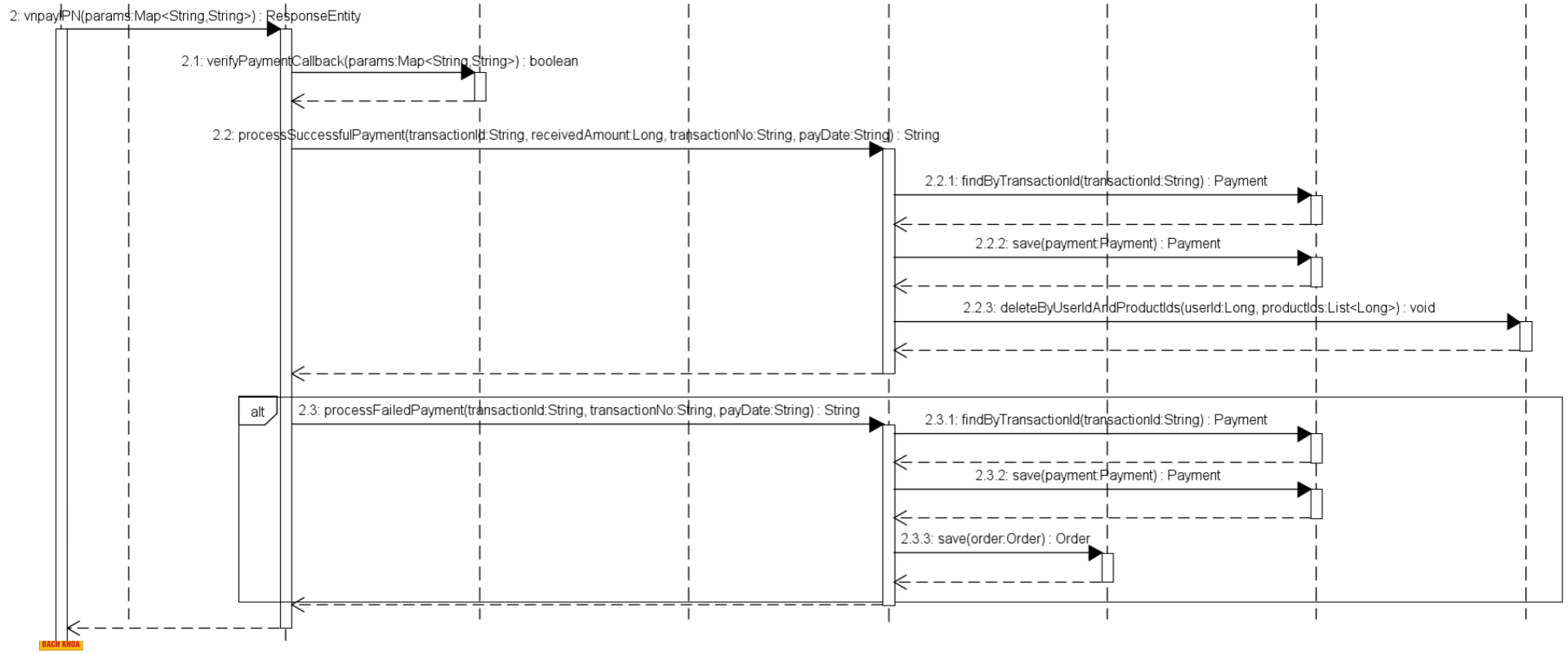
2.2. Core Workflows

- **Trigger:** Customer is redirected back to the system after payment
- **Main Purpose:** Verify the transaction results on the user interface.



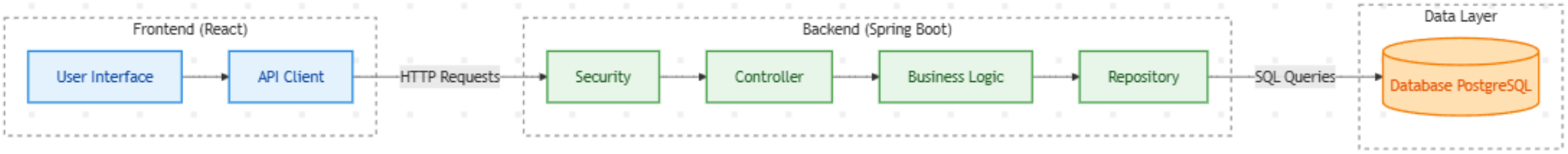
2.2. Core Workflows

- **Trigger:** Server-to-Server call directly from VNPay to Backend
- **Main Purpose:** The authoritative flow to update the persistent database state
  - **If Success:** Marks payment as PAID and clears cart
  - **If Failed:** Marks as FAILED, restores reserved stock, and cancels order



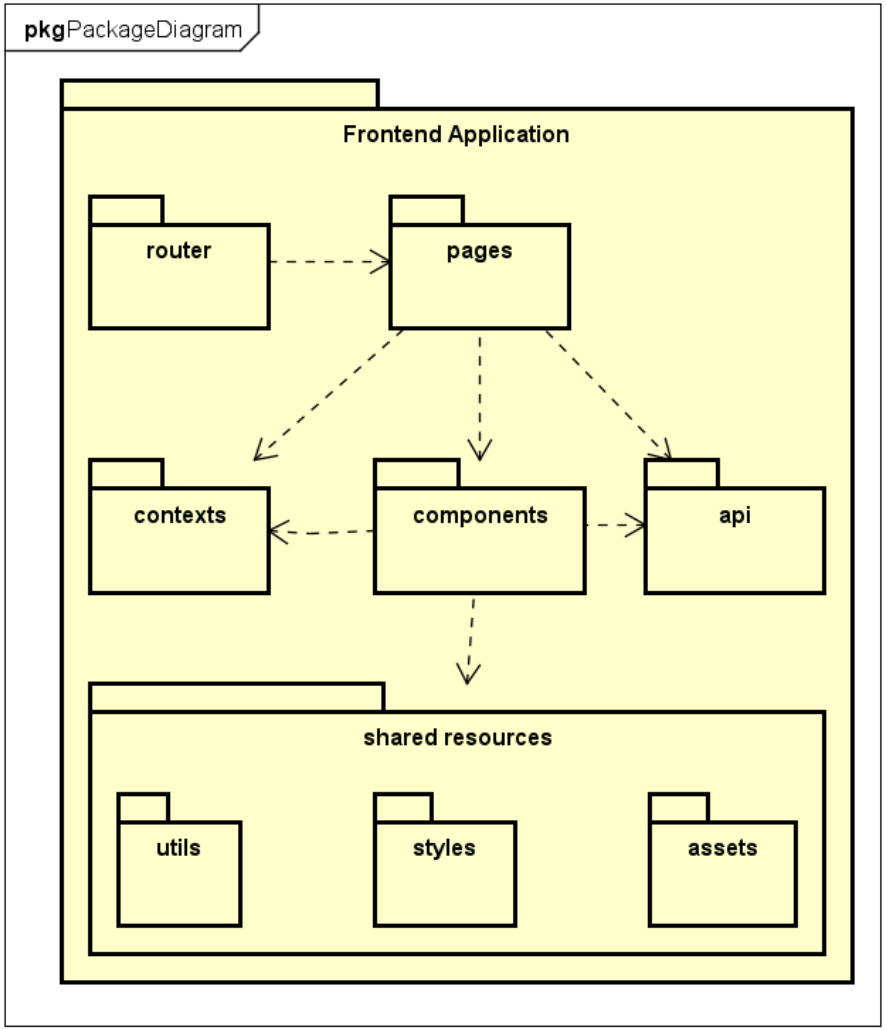
2.3. Architecture Design

- **Architecture Model:** Three-tier client-server architecture
  - Presentation layer **(React)**
  - Application layer **(Spring Boot)**
  - Data layer **(PostgreSQL)**



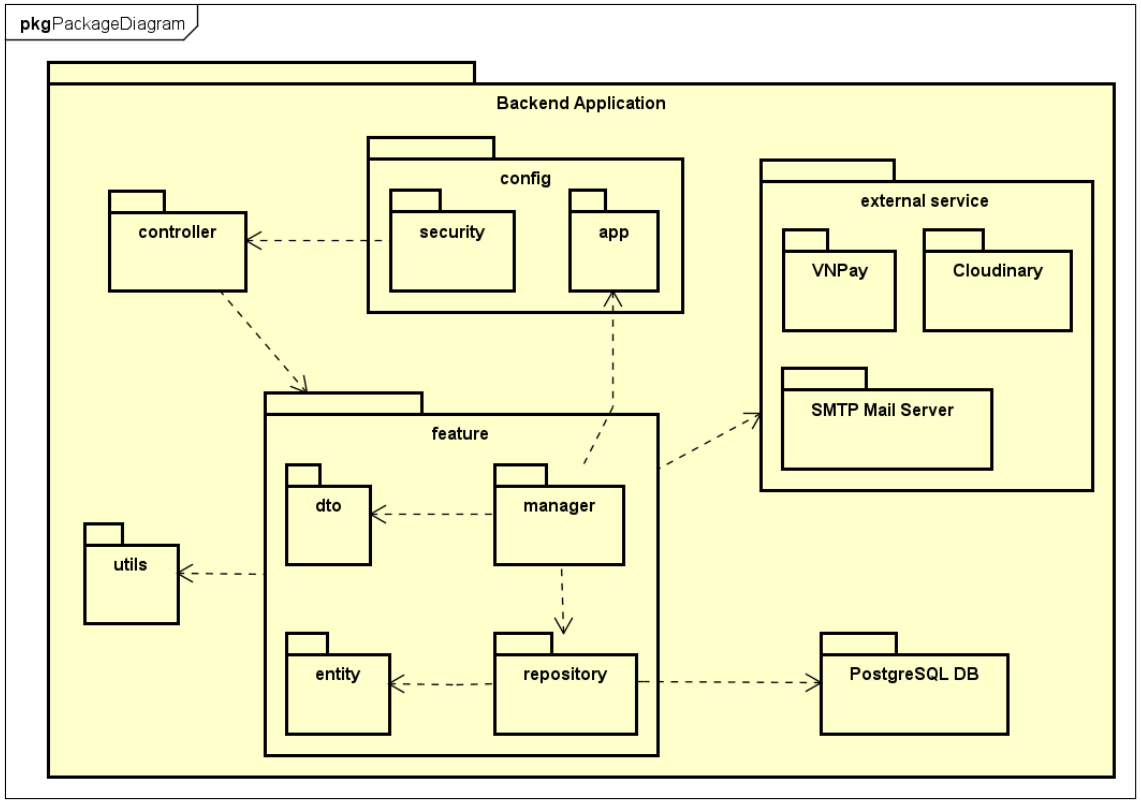
2.3. Architecture Design

Package	Purpose
router	Navigation & access control
pages	Role-based main screens
contexts	Global state management
components	Reusable UI elements
api	Backend communication



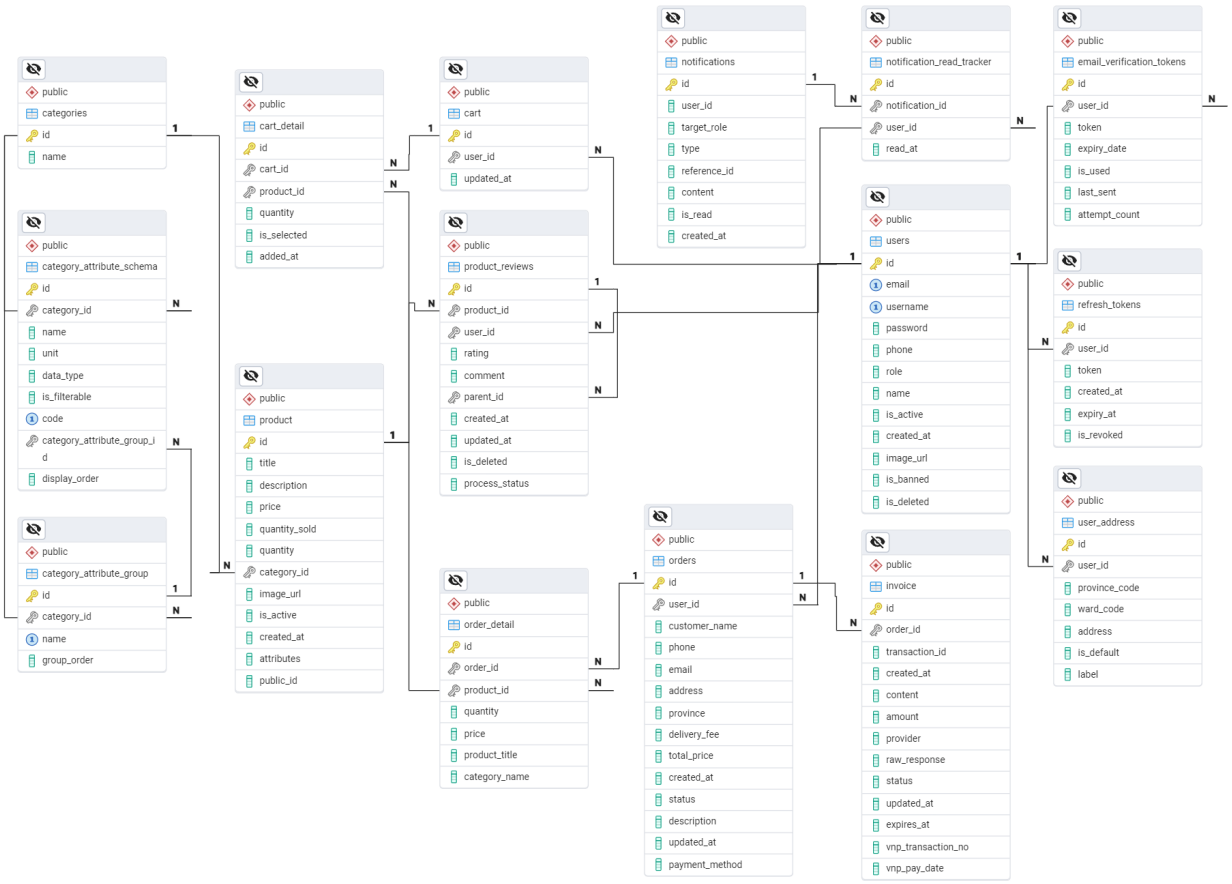
2.3. Architecture Design

Package	Purpose
config	Security & app settings
controller	API endpoints
manager	Core business logic
dto	Data transfer objects
repository	Database queries
entity	Database models



2.4. Database design

- **PostgreSQL**
- **16 tables**

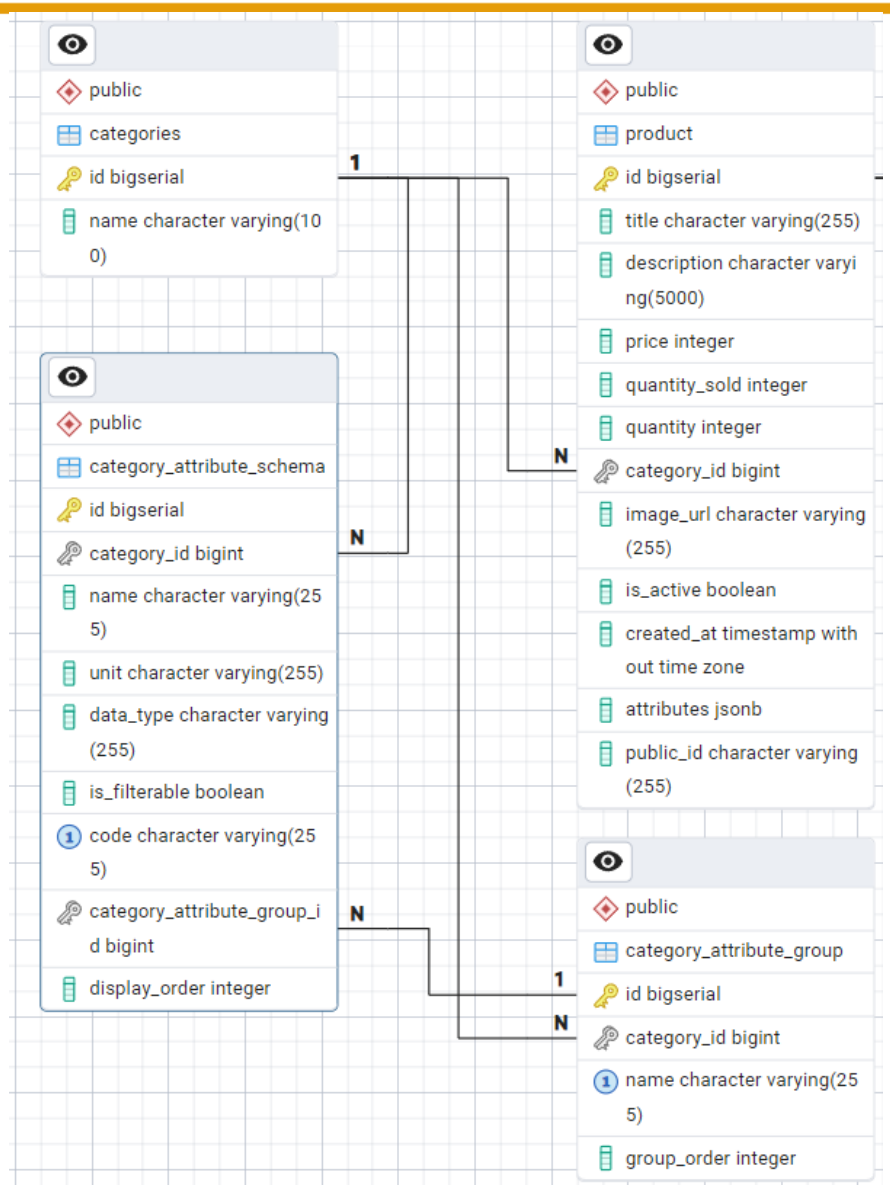


2.4. Database design

Product Schema Group:

- **categories**: Base product classification.
- **category_attribute_schema:** Defines validation rules & data types for each specification.
- **category_attribute_group**: Logical grouping for UI rendering.
- **product**: Stores fixed fields + **attributes (jsonb)** for flexible specification values.

→ Provides a unified, flexible structure that adapts seamlessly to any product category without schema modifications.



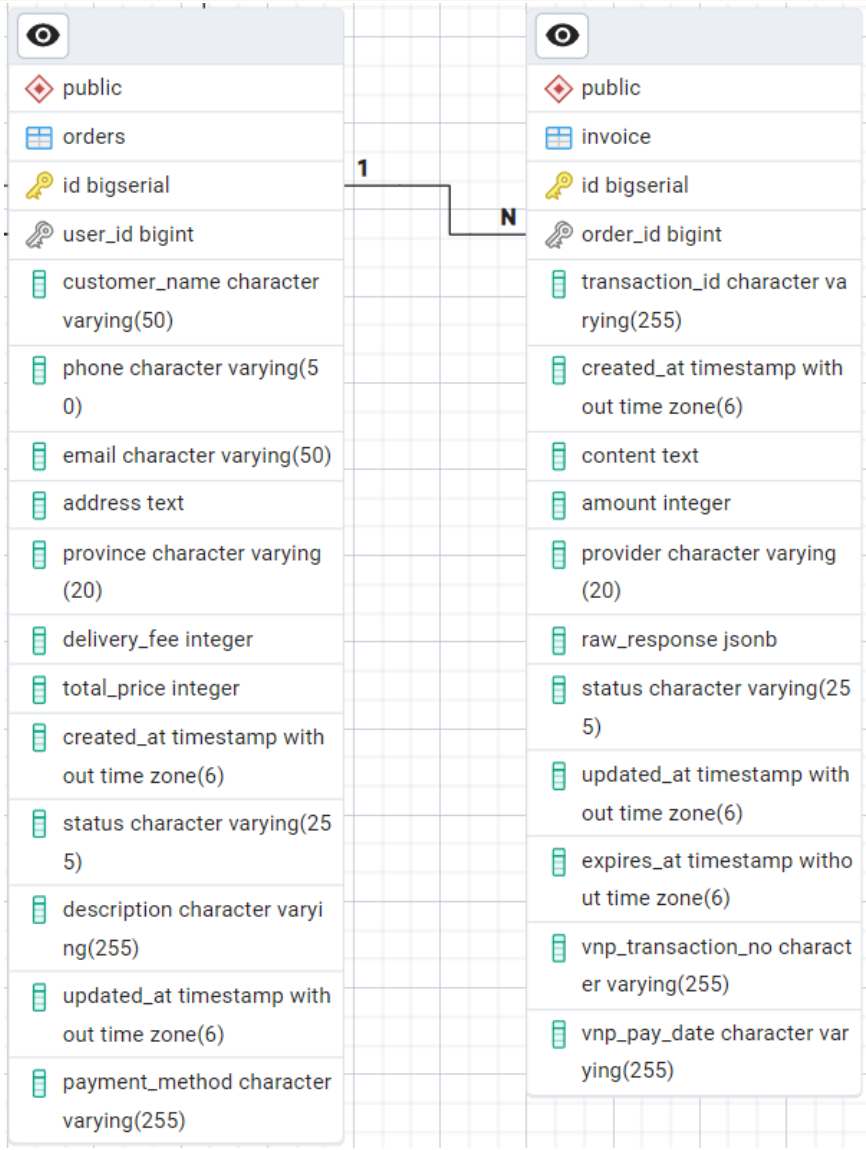
2.4. Database design

Order & Payment Schema Group:

- **orders**: Stores core purchase data.
- **invoice**: Stores billing records and payment details.

Dual-State Mechanism:

- **orders.status**: Tracks logistical workflow (fulfillment, shipping).
- **invoice.status**: Tracks financial workflow (transaction outcome).



3. Solutions & Implementation Results

3.1. Dynamic Product Attributes

3.2. VNPay Timeout Recovery

3.3. Memory-Bounded Excel Product Import

3.4. Implementation Results

3.1. Dynamic Product Attributes

- **Problem:**
  - Varying technical specifications across product categories.
  - **Fixed columns:** Inefficient, leading to sparse tables with many NULL values.
  - **EAV model:** Flexible, but increases records and requires too many JOINs, making query logic harder to understand and maintain.
- **Proposed Solution:**
  - **Value Storage:** PostgreSQL JSONB stores dynamic attributes efficiently in a single column.
  - **Schema Definition:** Relational tables define strict rules & data types (Number, Text, Boolean, List).
  - **Optimization:** GIN Indexing on JSONB optimizes the filtering process.

→ **Design Impact:** Achieves NoSQL-like flexible querying directly within an RDBMS, enabling dynamic category scalability and no need to create new tables.

3.2. VNPay Timeout Recovery

- **Problem:**
  - **Gateway Abandonment:** Customers frequently abandon transactions mid-way on the VNPay gateway → No IPN callback triggered.
  - **Data Inconsistency:** The system leaves orders stuck in a permanent PENDING state and reserved inventory locked indefinitely, preventing other users from purchasing.
- **Proposed Solution:**
  - **Background Monitor:** Implements a Scheduled Job to detect expired PENDING payments.
  - **Automated Cleanup:** Automatically marks payment as FAILED, cancels the order, and restores reserved stock.
  - **Data Consistency:** Recovery operations execute within a single database transaction.

→ **Design Impact:** Eliminate manual management tasks and virtual stock lockups, ensuring the accuracy and consistency of inventory.

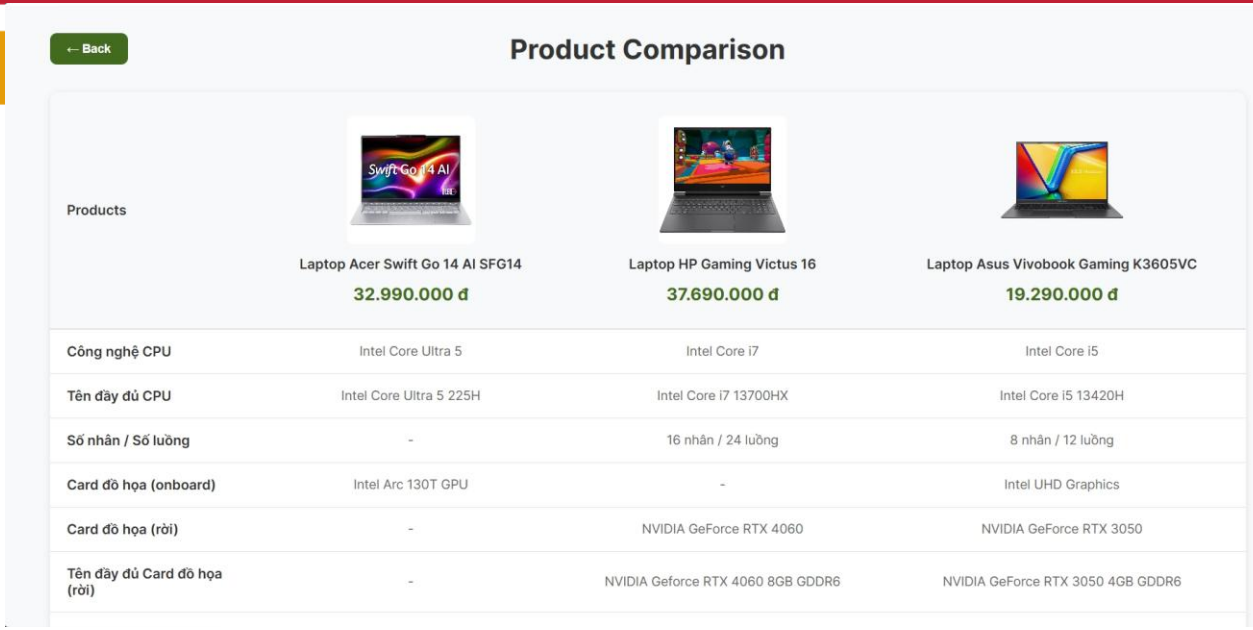
3.3. Memory-Bounded Excel Product Import

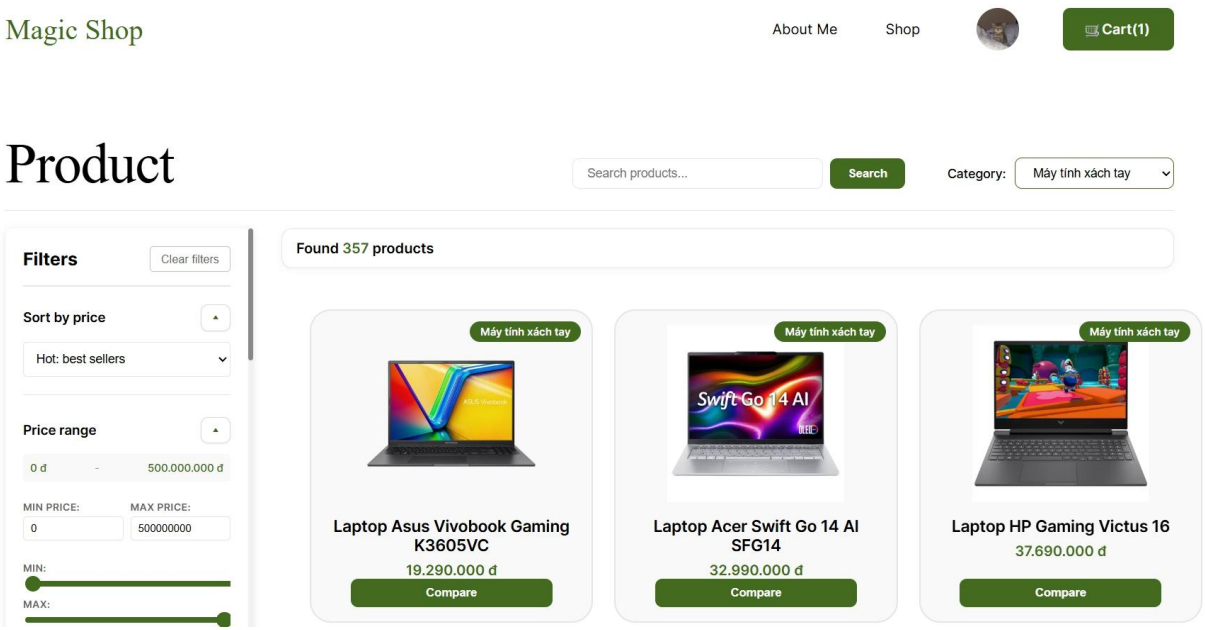
- **Problem:**
  - Loading large Excel files entirely into RAM risks Out-of-Memory (OOM) crashes.
  - Sequential image uploads to Cloudinary create severe I/O bottlenecks.
  - Dynamic attributes require strict validation against the category schema.
- **Proposed Solution:**
  - **Memory-Bounded Processing:** Uses EasyExcel for streaming-based read (row-by-row) and batch saving.
  - **Performance Optimization:** Implements asynchronous image uploads to Cloudinary concurrently within each batch.
  - **Strict Validation:** Separates fixed fields and validates dynamic attributes against the category_attribute_schema table (number, boolean, text, list).

→ **Design Impact:** Eliminates OOM risks, significantly accelerates data entry, and ensure the structural integrity of imported product catalogs.

3.4. Implementation Results

Hardware Specification Comparison

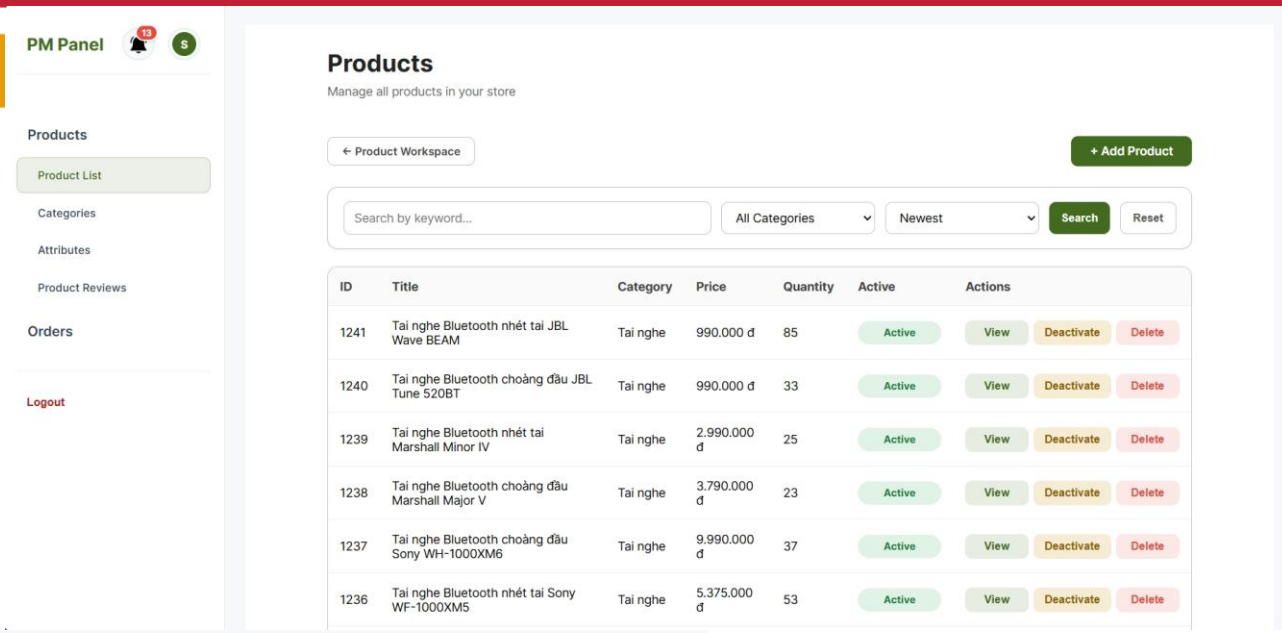


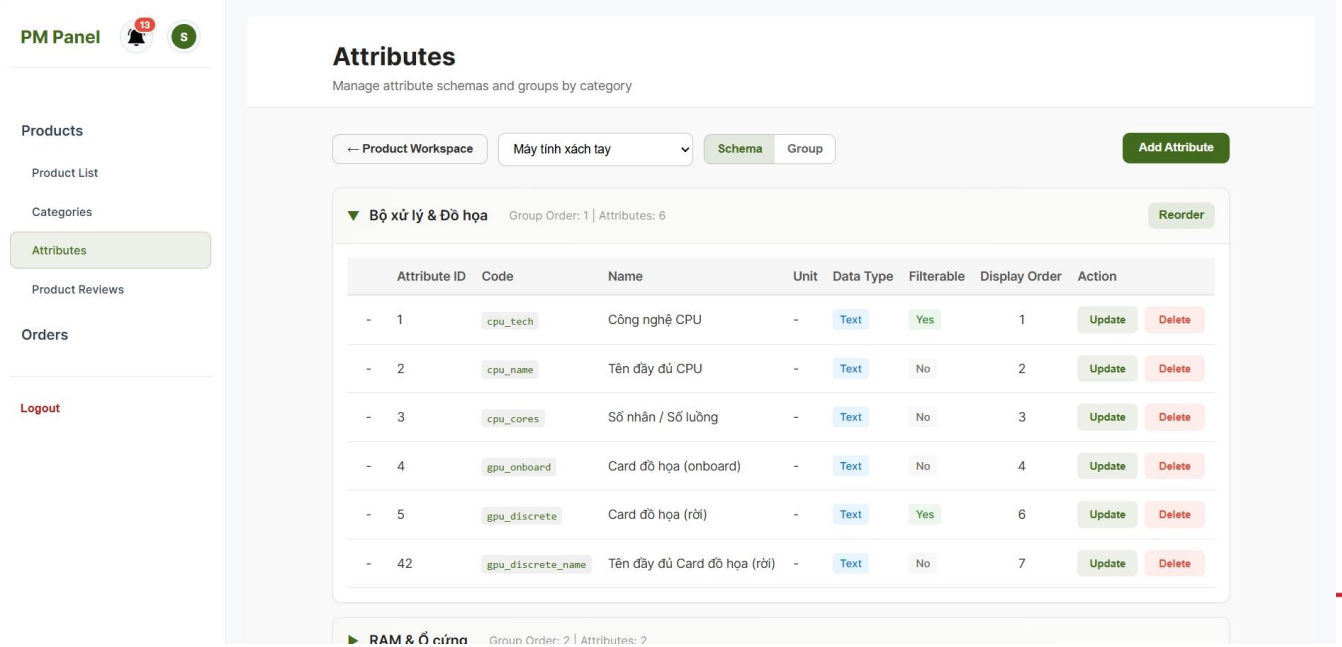


Product Discovery & Dynamic Filtering

3.4. Implementation Results

Product Management

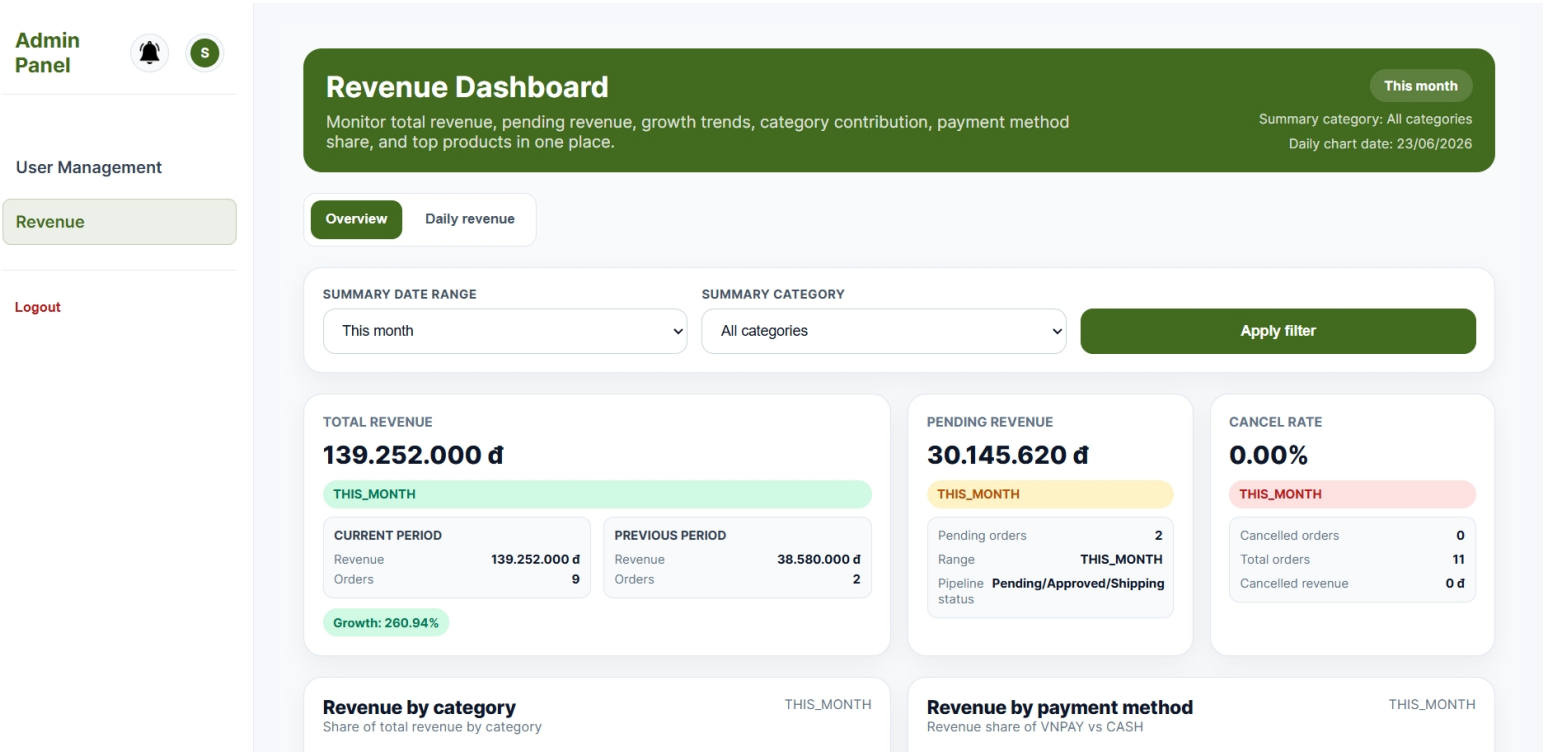




Dynamic Attribute Schema Management

3.4. Implementation Results

Admin Revenue Dashboard



4. Conclusion and Future Work

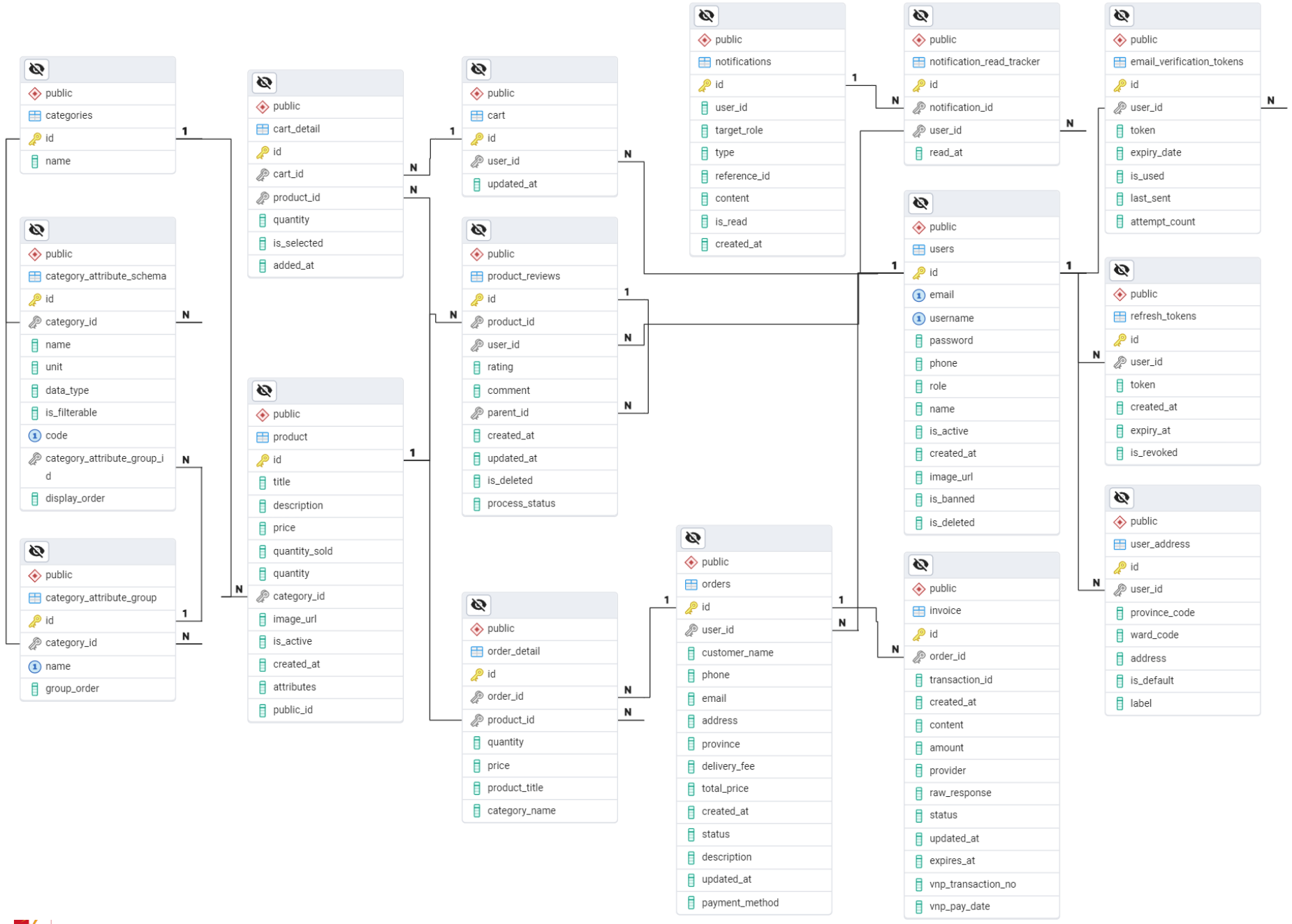
4. Conclusion and Future Work

- **Achievements :**
  - Developed a complete full-stack tech e-commerce system (Customer Portal & Management Dashboard)
  - **Successfully implemented core technical solutions**: Dynamic attributes (JSONB), VNPay timeout recovery, and Memory-bounded Excel import
  - **Integrated external services:** VNPay, Cloudinary, Email OTP.
- **Limitations:**
  - Tested mainly in development environments, lacks real-world load testing
  - Missing advanced business features (promotions, third-party logistics,...)
- **Future Work:**
  - Production deployment with CI/CD pipelines and centralized monitoring.
  - Integrate third-party delivery providers (Dynamic shipping fees) and promotion/voucher systems.
  - Enhance product discovery with Semantic Search, Recommendation Systems and AI Chatbot.

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !


public
categories
id
name
public
category_attribute_schema
id
category_id
name
unit
data_type
is_filterable
code
category_attribute_group_id
display_order
public
category_attribute_group
id
category_id
name
group_order
public
cart_detail
id
cart_id
product_id
quantity
is_selected
added_at
public
product
id
title
description
price
quantity_sold
quantity
category_id
image_url
is_active
created_at
attributes
public_id
public
cart
id
user_id
updated_at
public
product_reviews
id
product_id
user_id
rating
comment
parent_id
created_at
updated_at
is_deleted
process_status
public
order_detail
id
order_id
product_id
quantity
price
product_title
category_name
public
notifications
id
user_id
target_role
type
reference_id
content
is_read
created_at
public
orders
id
user_id
customer_name
phone
email
address
province
delivery_fee
total_price
created_at
status
description
updated_at
payment_method
public
notification_read_tracker
id
notification_id
user_id
read_at
public
users
id
email
username
password
phone
role
name
is_active
created_at
image_url
is_banned
is_deleted
public
invoice
id
order_id
transaction_id
created_at
content
amount
provider
raw_response
status
updated_at
expires_at
vnp_transaction_no
vnp_pay_date
public
email_verification_tokens
id
user_id
token
expiry_date
is_used
last_sent
attempt_count
public
refresh_tokens
id
user_id
token
created_at
expiry_at
is_revoked
public
user_address
id
user_id
province_code
ward_code
address
is_default
label